```
"""
Water Jug Problem Implementation in Python

The water jug problem is a classic problem in which you have two jugs with
different capacities and an unlimited supply of water. The objective is to
measure out a specific amount of water using the two jugs. You can fill
a jug completely, empty a jug, or pour water from one jug to the other until
either the first jug is empty or the second jug is full.
"""

from collections import deque

def water_jug_bfs(cap1: int, cap2: int, target: int):
    """
    Solve the water jug problem using breadth-first search (BFS).

    Args:
        cap1 (int): Capacity of the first jug.
        cap2 (int): Capacity of the second jug.
        target (int): Target amount of water to measure.
    """
    visited = set() # To keep track of visited states
    queue = deque() # To manage the states to explore

    queue.append((0, 0))  # Start with both jugs empty
    visited.add((0, 0)) # Mark the initial state as visited

    # To keep track of the parent states for solution reconstruction
    parent_map = {}

    # BFS loop
    while queue:
        jug1, jug2 = queue.popleft() # Get the current state of the jugs

        # Check if we have reached the target amount in either jug
        if jug1 == target or jug2 == target:
            print("Solution found:")
            print_solution(parent_map, (jug1, jug2)) # Print the solution path
            return

        next_states = [
            (cap1, jug2),  # Fill jug1
            (jug1, cap2),  # Fill jug2
            (0, jug2),     # Empty jug1
            (jug1, 0),     # Empty jug2
            # Pour from jug1 to jug2
            (jug1 - min(jug1, cap2 - jug2), jug2 + min(jug1, cap2 - jug2)),
            # Pour from jug2 to jug1
            (jug1 + min(jug2, cap1 - jug1), jug2 - min(jug2, cap1 - jug1))
        ]

        # Explore the next states
        for state in next_states:
            if state not in visited: # Check if the state has not been visited
```

```
                visited.add(state) # Mark the state as visited
                queue.append(state) # Add the new state to the queue for further exploration
                parent_map[state] = (jug1, jug2)  # Keep track of the parent state

    # If we exhaust the queue without finding the target, print that no solution exists.
    print("No solution found.")


def print_solution(parent_map: dict, state: tuple):
    """
    Print the solution path from the initial state to the target state.

    Args:
        parent_map (dict): A mapping of states to their parent states.
        state (tuple): The target state to trace back from.
    """
    solution = [] # To store the solution path
    # Trace back the path from the target state to the initial state using the parent_map
    while state in parent_map:
        solution.append(state) # Add the current state to the solution path
        state = parent_map[state] # Move to the parent state

    solution.append(state)  # Add the initial state
    # Reverse the solution path to get the correct order from
    # initial state to target state
    solution.reverse()

    # Print each step in the solution path
    for step in solution:
        print(step)


def main():
    """
    Main function to run the water jug problem solver.
    """
    cap1 = int(input("Enter capacity of Jug 1: "))
    cap2 = int(input("Enter capacity of Jug 2: "))
    target = int(input("Enter the target amount of water: "))

    water_jug_bfs(cap1, cap2, target)


if __name__ == "__main__":
    main()




# Output:
# Enter capacity of Jug 1: 4
# Enter capacity of Jug 2: 3
# Enter the target amount of water: 2

# Solution found:
```

```
# (0, 0)
# (0, 3)
# (3, 0)
# (3, 3)
# (4, 2)
```